NOVA IMS INFORMATION MANAGEMENT SCHOOL

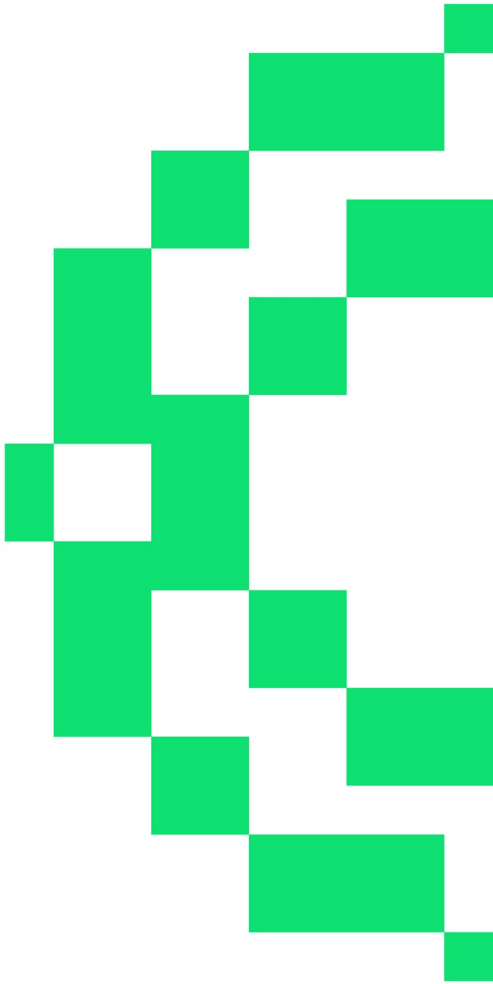
`while` and `break`: more on loop control

Computation I · Lecture 08

Karina Brotto Rebuli
krebuli@novaims.unl.pt

Two loops that would not stop when asked

Last week three tries ran out before the player was right, and the search for the first negative could not stop once it found one — it ran on and reported them all. Today one loop repeats until the answer is right, and one word leaves a loop the moment its work is done.



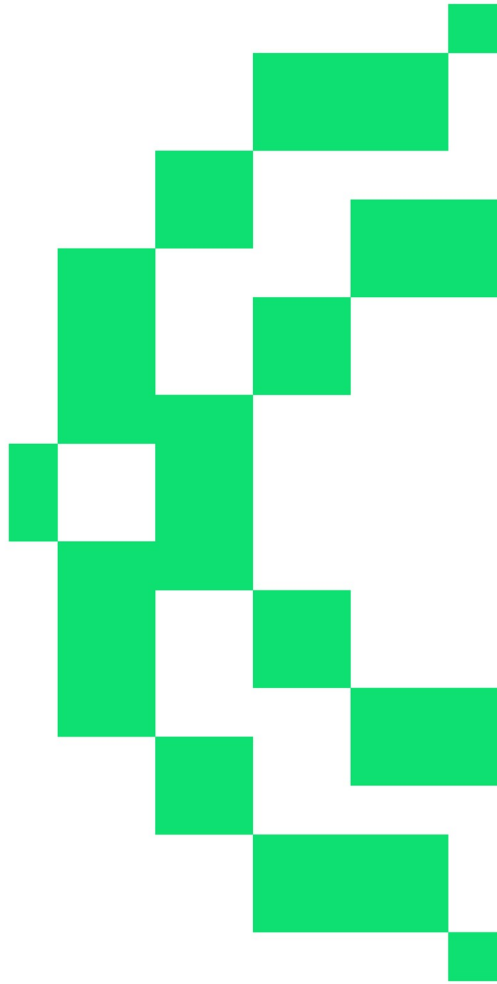
```
"""

for knows how many times.

while knows when to stop.

break stops it right now.

"""
```

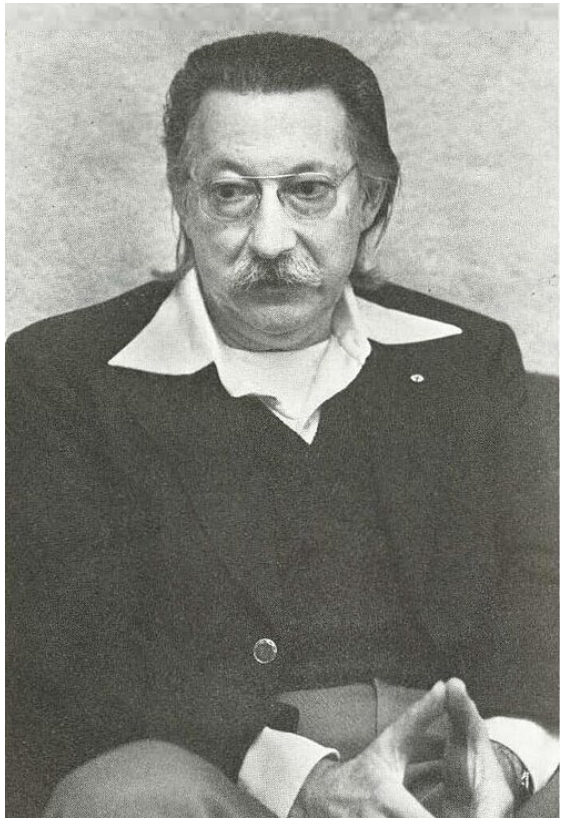


Joseph Weizenbaum

1966 — How to program a process whose number of steps is not fixed when it begins.

The problem: he built ELIZA, a program that answered a person one line at a time, and it could not know in advance how many lines the exchange would take. A conversation lasts exactly as long as the person keeps talking.

Try it: `masswerk.at/elizabot/eliza.html`



Wikimedia Commons

Three questions today

How do you repeat something without deciding how many times?

How do you leave a loop the instant its work is done?

`while` or `for` — which one should you write?

01.

Repeat while it holds

The loop that sets its own length

LIVE

The game that ran out of tries

Exercise 9, replayed — three guesses, then it stops

```
secret = 7
for attempt in range(3):
    guess = int(input("Guess: "))
    if guess == secret:
        print("You got it!")
```

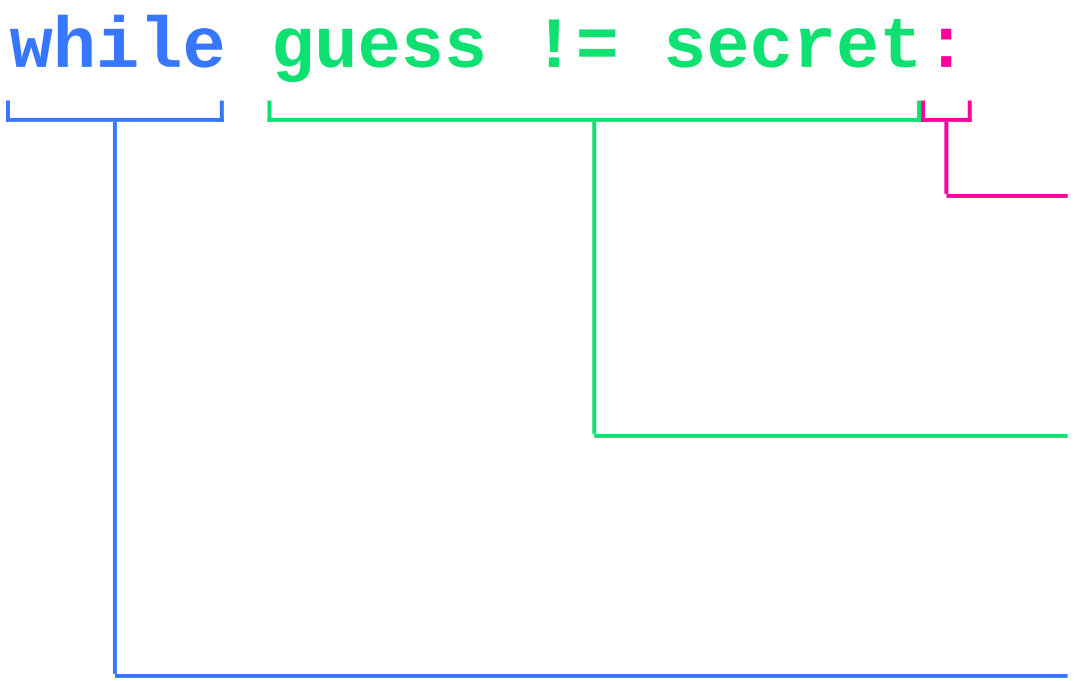
LIVE

The game that ran only the tries it needed

It runs only while the condition says so — no counting, just checking the condition

```
secret = 7
guess = int(input("Guess: "))
while secret != guess:
    guess = int(input("Try again: "))
print("You got it!")
```

The shape of a `while` loop



COLON, THEN BODY

The indentation defines the block that repeats.

THE CONDITION

Any expression that answers True or False — whatever its form. The variable it tests needs a value before the loop, and the body must change it.

KEYWORD

Starts the loop — and asks its question before every turn.

KEEP THIS It repeats while the condition is True, and stops at the first False.

A `while` repeats while a condition is `True`

The engine is a slice — nothing counts, and it still stops

```
word = "python"
while len(word) > 3:
    print(word)
    word = word[:-1]
```

```
>>> python
>>> pytho
>>> pyth
```

`while` trap 01: forget to initialise it, and it never starts

The condition raises a runtime error — a cheaper mistake than the next one

```
while x < 3:
    print(x)
```

```
NameError: name 'x' is not defined
```

`while` trap 02: forget to move it, and it never stops

The condition stays true forever — do NOT run it!

```
x = 0
while x < 3:
    print(x)
```

```
>>> 0
>>> 0
>>> 0
>>> ...
```

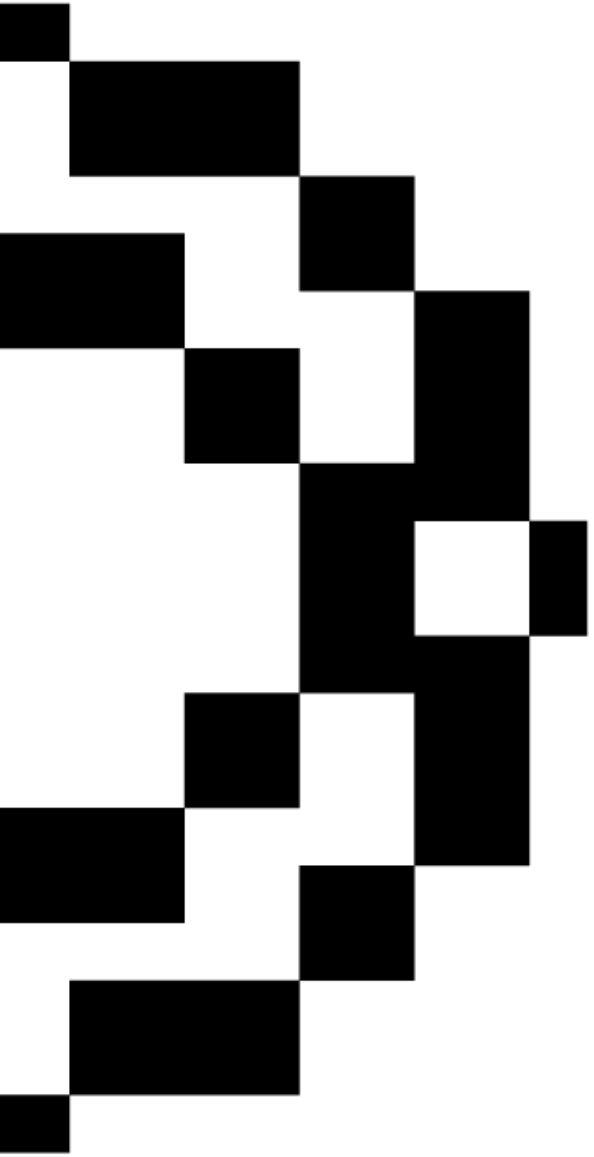
LIVE

What does it print, and what is the value of n when it finishes?

Write down a countdown starting at 10 and decreasing by 3 each time — on paper before committing

```
n = 10
while n > 0:
    print(n)
    n = n - 3
print(n)
```

```
>>> 10
>>> 7
>>> 4
>>> 1
>>> -2
```



02.

Leave when you’re done

`break` steps out of the loop

LIVE

The same countdown, leaving early

`if` checks the next step — `break` leaves before it overshoots

```
n = 10
while n > 0:
    print(n)
    if n - 3 < 0:
        break
    n = n - 3
print(n)
```

- the exit condition
- leaves the loop now
- skipped code
- outside the loop: still runs

Loop on purpose, `break` when it's valid

`while` and `break` together: the guard that insists

```
while True:
    age = input("Age: ")
    if age.isdigit():
        break
    print("Please type a number.")
age = int(age)
```

The first negative, and only the first

Exercise 10, resolved — stop at the match

```
values = [4, 7, -2, 9, -5]
for i in range(len(values)):
    if values[i] < 0:
        print("Negative at", i)
        break
```

LIVE

Stop when the user guesses — at most three attempts

Exercise 9 with `for` and `break` — it leaves on the right guess, and the max count is fixed

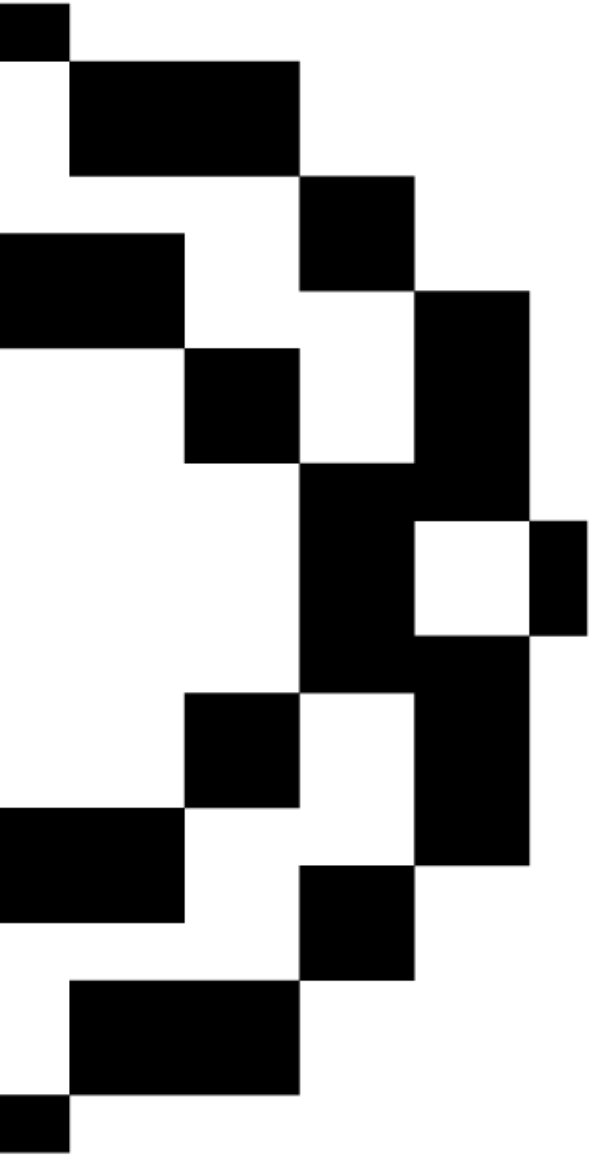
```
secret = 7
for attempt in range(3):
    guess = int(input("Guess: "))
    if guess == secret:
        print("You got it!")
        break
```

LIVE

Max 3 attempts, asking only until it’s right

Exercise 9, resolved — now respecting the fixed maximum of attempts, but controlling the counter manually

```
secret = 7
attempts = 0
while attempts < 3:
    guess = int(input("Guess: "))
    if guess == secret:
        print("You got it!")
        break
    attempts = attempts + 1
```



03.

The same job, two ways

`while` or `for` — you choose

Print 0 to 9 — `while` vs `for`

Who manages the counter?

WITH FOR

```
for i in range(10):
    print(i, end=" ")
```

```
>>> 0 1 2 3 4 5 6 7 8 9
```

THE SEQUENCE (`range` HERE) MANAGES THE COUNTER FOR YOU

WITH WHILE

```
i = 0
while i < 10:
    print(i, end=" ")
    i = i + 1
```

```
>>> 0 1 2 3 4 5 6 7 8 9
```

YOU MANAGE THE COUNTER YOURSELF

LIVE

Count the spaces — twice

Directly with `for`, by index with `while`

WITH FOR

```
s = input("Text: ")
count = 0
for c in s:
    if c == " ":
        count = count + 1
print(count)
```

WITH WHILE

```
s = input("Text: ")
count = 0
i = 0
while i < len(s):
    if s[i] == " ":
        count = count + 1
    i = i + 1
print(count)
```

Prefer the `for` when you can

Every `while` that just counts can be a `for`

`for`

based on sequences — the number of turns is already set when the loop begins

an object in memory
`list, str`

an iterable
`range()`

WHO MOVES IT — the sequence hands you each item.

HOW IT ENDS — when the sequence runs out.

`while`

based on the current state — a condition tested before every turn, as the loop runs; the number of turns is discovered, not set

any expression that answers True or False
`guess != secret` · `i < len(s)`

WHO MOVES IT — you start, test and step the variable yourself.

HOW IT ENDS — at the first False.

Your turn · Count the spaces in the first sentence

Change the program so that it counts only the spaces in the first sentence — everything before the first full stop. Write it twice, without string split(), like the counter itself: the `for` version with this lecture's `break`, and a `while` whose condition does the stopping. One run of the finished program, for example:

WITH FOR

```
s = input("Text: ")
count = 0
for c in s:
    if c == ".":
        break
    if c == " ":
        count = count + 1
print(count)
```

```
Text: Hi there. How are you?
1
```

WITH WHILE

```
s = input("Text: ")
count = 0
i = 0
while i < len(s) and s[i] != ".":
    if s[i] == " ":
        count = count + 1
    i = i + 1
print(count)
```

```
Text: Hi there. How are you?
1
```

Two tools, one habit

In one sentence each

while

repeats while a condition holds — you set the condition, not the number of turns

break

leaves the loop the moment its work is done — normally written inside an if

while or for

every counting while can be a for; reach for while when the number of turns is unknown

Joseph Weizenbaum

1966

Every loop today ran a number of times no one set in advance
— that was his whole problem, and today it is two keywords:
while repeats while it holds, break leaves when done.

Before the next theoretical lecture

On your own

Think Python — Sections 5.4 `if` statements · 5.5 The `else` clause · 7.3 Updating variables.

One habit — check that every loop has a condition that ends it, or a break statement.

Exercise 1 · Predict what it prints

Read the `while` below and write each printed line, and the value of `n` when the loop stops — trace it by hand. On paper, before any run. Then run it and compare.

```
n = 5
while n > 0:
    print(n)
    n = n - 2
```

Exercise 2 · Predict what it prints

Read the program below and write the number it prints, and how many turns the loop took. On paper, then run it and compare.

```
total = 0
i = 1
while i <= 4:
    total = total + i
    i = i + 1
print(total)
```

Exercise 3 · The odd numbers, one per line

Write a `while` loop that prints the odd numbers from 1 to 9, one per line, exactly as the output below shows. Use this lecture's `while` — the counter, the test and the step are all yours to write; that is the point of this week.

```
>>> 1
>>> 3
>>> 5
>>> 7
>>> 9
```

Exercise 4 · Fix the program

This program should stop at the first zero. It runs and raises nothing, yet it reports both zeros. Say what is missing, where it belongs, and write the fix. Everything in it is this lecture's material.

```
values = [4, 0, 7, 0]
for v in values:
    if v == 0:
        print("found zero")
```

Exercise 5 · Ask until the answer is yes

Write a program that keeps asking `Continue?` until the user types exactly `yes`, then prints `ok`. Use this lecture's `while` — the question itself is the condition. One run of the finished program, for example:

```
Continue? no
Continue? maybe
Continue? yes
ok
```

Exercise 6 · Predict what it prints

Read the loop below and write exactly which numbers print, and say at which value the loop ends. On paper, before any run, then run it and compare.

```
for n in [3, 6, 9, 12]:
    if n > 8:
        break
    print(n)
```

Exercise 7 · The first number over 100

Write the loop that prints the first number over 100 in the list — that number and nothing after it. Start from the list given in the panel. Use this lecture's `break`, inside week 6's `for`.

```
values = [40, 80, 120, 200]
# your loop here
```

```
>>> 120
```

Exercise 8 · Predict what it prints

Read the two loops below — a `while`, then the `for` you already know. On paper: write every line the program prints, in order. Then run it and compare.

```
i = 0
while i < 3:
    print(i)
    i = i + 1
for i in range(3):
    print(i)
```

Exercise 9 · A program that chats

Write a program that chats: it greets the user, then keeps reading answers with `input` until the user types `bye`, and then says goodbye. Pick two or three words to react to: when an answer contains one of them (week 3's `in`), reply with a matching question; otherwise reply with one neutral line that keeps the conversation going. Use this lecture's `while` — the length of the conversation is the user's choice, not yours.

One run of the finished program, for example:

```
Hello, traveller. Where would you like to go?
> a quiet beach in the Algarve
Sun and sea! Which beach?
> maybe the mountains in winter
Fresh air! Hiking or skiing?
> bye
Goodbye, and have a good trip!
```

Recap 1 · Predict what it prints

Read the program below and write exactly what it prints — every line, in order. On paper, before any run. Then run it and compare.

```
values = [3, 5, 2, 9, 4]
total = 0
i = 0
while i < len(values):
    if values[i] > 7:
        break
    total = total + values[i]
    i = i + 1
print(total, i, values[i])
```

Recap 2 · Choose the fragment

The program should print the letters of word, one per line, walking the string by its index — and then print i. It starts with word = "loop" and i = 0, and ends with print(i). Exactly one of the four fragments below does the job. Choose it — and say what goes wrong in each of the other three.

```
# A
while i < len(word):
    print(word[i])
    i = i + 1
# B
while i <= len(word):
    print(word[i])
    i = i + 1
```

```
# C
while i < len(word):
    i = i + 1
    print(word[i])
# D
while i < len(word):
    print(word[i])
    i + 1
```

Recap 3 · Debug

The program should ask for numbers until the user types 0, then print how many numbers were typed and their sum. It runs, but it never stops — not even after a 0. Find every line to blame, say why each one keeps the loop going, and write the fixes.

```
total = 0
count = 0
n = input("Number: ")
while n != 0:
    total = total + int(n)
    count = count + 1
n = input("Number: ")
print(count, total)
```

Exercise 10 · Print the 2 times table ★

Run the loop in the panel — the 2 times table, one line per product; it works. Now print every table, from 1 to 10, using only what has been taught so far — and watch what your program starts to look like.

```
for n in range(1, 11):
    print(2 * n)
```

NOVA IMS INFORMATION MANAGEMENT SCHOOL

```
input("Questions? ")
```

Acreditações e Certificações da NOVA IMS

eduniversal · QS STARS · ABET Computing Accreditation Commission · iSchools · EMOS · UNIGIS · USGIF · Erasmus Mundus Programme Education and Culture DG · Lapcer · IQNet

Cofinanciado por

Portugal 2030 · Lisboa 2030 · Co-funded by the European Union · fct Fundação para a Ciência e a Tecnologia · PRR Plano de Recuperação e Resiliência · República Portuguesa · Financiado pela União Europeia NextGenerationEU

© Karina Rebuli, 2026